

PPAU VLSI - Laboratorium 2

Podczas tych zajęć poddane będą analizie rozrzuty technologiczne, jakie występują w zintegrowanych układach elektronicznych. W pierwszych krokach będą analizowane rozrzuty prądu klasycznego źródła prądowego i parametry od jakich one zależą, a następnie zbudowany będzie 5-cio bitowy przetwornik cyfrowo-analogowy DAC z ważonymi prądami.

Wykorzystywane środowisko to IIC-OSIC-TOOLS ([github](#)), a używany PDK to sky130A od SkyWater Foundries ([pdk-documentation](#)). Sama biblioteka sky130 ma wiele przykładowych komórek bazowych jak i testów dla różnych układów. Dostęp do wszystkich można znaleźć w różnych folderach tej biblioteki, gdzie przykładowo `sky130_tests/` zawiera wiele przykładowych bloków symulacji dla różnych układów, w tym wykorzystywane na tym laboratorium analizy parametryczne jak Monte Carlo. Ponieważ symulacje nie są kontrolowane za pomocą interfejsu graficznego, należy zapoznać się ze składnią poleceń ngspice (do sprawdzania składni i przykładów - [manual](#))

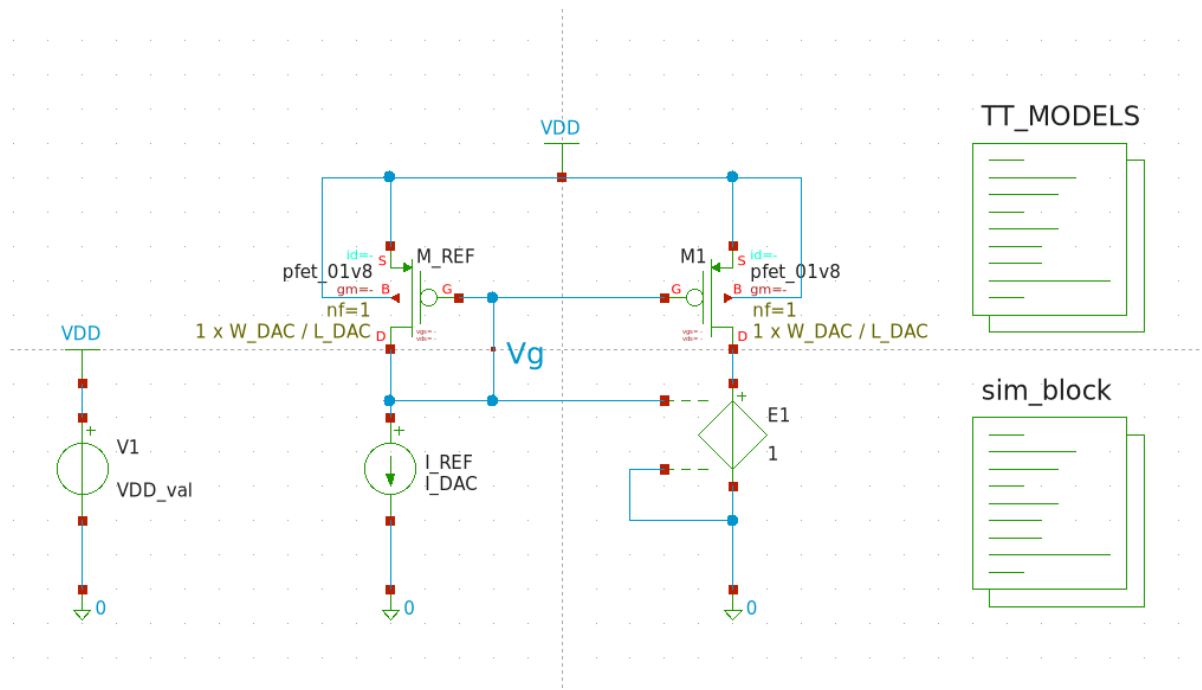
1. Rozrzuty w zintegrowanych układach logicznych

1.1. Przygotuj schemat układu źródła prądowego z Rys. 1.1. Jako tranzystorów użyj **pfet_01v8.sym** z biblioteki **sky130_fd_pr** (fd - The SkyWater Foundry, pr - Primitive Cells; więcej o nazewnictwie w sky130 PDK można znaleźć [tutaj](#)), a dla całego schematu użyj parametrów:

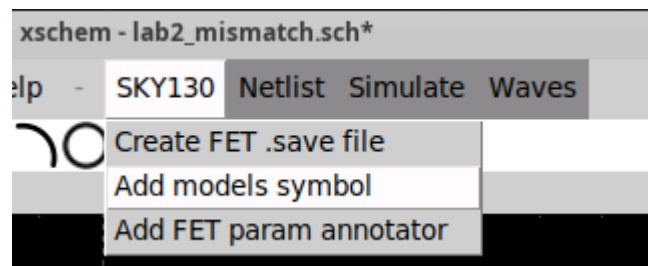
- szerokość i długość kanału mosfetów -> odpowiednio **W_DAC** i **L_DAC**
- Źródłu prądowemu -> parametr **I_DAC**
- Źródłu napięcia -> parametr **VDD_val**

Aby w realizowanych analizach wyeliminować błąd systematyczny związany z różnymi napięciami Vds tranzystorów, użyj źródła napięciowego **vcvs.sym** z domyślnie dostępnej biblioteki **devices** z ustawionym wzmocnieniem 1. Elementy widoczne na schemacie takie jak źródła prądowe, napięciowe, symbole zasilania, uziemienia czy bloki kodu także można znaleźć w tej bibliotece. Napięcie zasilania VDD powinno być równe 1.8V.

Wykorzystany blok kodu o nazwie TT_MODELS to gotowy blok, który jest łatwo i szybko dostępny wybierając na pasku menu `SKY130 -> Add model symbol` (Rys 1.2.) i dla parametrów typowych nie musi być edytowany.



Rys 1.1. Schemat źródła prądowego.



Rys 1.2. Dodanie bloku kodu zawierającego bibliotekę.

1.2. Następnie sprawdzimy punkt pracy tranzystorów PMOS.

Dla trzech różnych wymiarów tranzystorów W / L odpowiednio:

- 0.24 / 0.18 [μm]
 - 5 / 2 [μm]
 - 2 / 5 [μm]
- oraz dwóch różnych prądów I_{DAC} :
- 100 [nA]
 - 10 [μA]

Wyznacz napięcia V_{gs} i V_{th} tranzystora M1. Uzyskane wyniki zanotuj w sprawozdaniu.

1.3. Aby przeprowadzić analizy Monte Carlo musisz wskazać środowisku symulacyjnemu modele elementów technologicznych, w których będą występowały parametry związane z rozrzutami technologicznymi. Modele te są identyczne jak te stosowane do tej pory, a różnią się jedynie dodatkowymi zmiennymi odpowiedzialnymi za rozrzuty.

W tym celu wystarczy zmienić parametr, z którym wywoływana jest biblioteka w bloku TT_MODELS. W lini, w której załączana jest biblioteka (.lib \$::SKYWATER_MODELS/sky130.lib.spice tt) zamień `tt` na wybrany corner. Na razie zastąp to przez `tt_mm`. Warto też zobaczyć co dokładnie znajduje się w załączanym pliku.

```
cd /foss/pdks/sky130A/libs.tech/ngspice
view sky130.lib.spice
```

1.4. W środku pliku `sky130.lib.spice` można zauważyć dużo zdefiniowanych cornerów wraz z informacją, które biblioteki z tego folderu są dołączane. Dla każdego corneru ustawiane są dwa parametry - przejrzyj te cornery, szczególnie **tt**, **tt_mm**, **mc**, i zastanów się do czego są te parametry i co oznaczają.

1.5. W kolejnym kroku skonfiguruj symulację Monte Carlo. W Xschem jest to realizowane przez załączanie odpowiednich plików bibliotecznych (blok TT_MODELS) oraz pętli kodu ngspice. Przykład takiej pętli jest dostępny na schemacie `sky130_tests/montecarlo_mismatch_sim.sch` w bloku kodu "NGSPICE". Alternatywą dla bloku TT_MODELS jest dostępny symbol `sky130_fd_pr/corner.sym`, w którym zmieniając wyłącznie parametr *corner* można dołączyć odpowiednie pliki. Blok TT_MODELS ze względu na to że zawiera pole kodu zamiast pojedynczego parametru, przez co możemy je wykorzystać do definicji idealnego modelu - nie podlegającego rozrzutom.

Dane można zbierać do plików za pomocą `echo [text and vectors] >> [file_path/filename]` lub polecenia `wrdata` (więcej w ngspice manual). Aby wyświetlić wyniki w formie graficznej zebrane dane w plikach można wyeksportować i wyplotować za pomocą wbudowanego `gnuplot` bezpośrednio z otwartej konsoli dockera lub ngspice'a. Dzięki dostępowi do konsoli dockera można pisać skrypty .sh, .tcl, które z zapisanych danych będą tworzyć wykresy i zapisywać je do plików graficznych.

1.6. Rozrzuty w PDK Sky130 są sterowane globalnie poprzez parametry `mc_mm_switch` i `mc_pr_switch`. Jeśli chcesz, aby dany tranzystor (np. lustro referencyjne `M_REF`) nie podlegał zmianom związanym z mismatchem czy procesem należy stworzyć idealny model naszego elementu, który nie będzie brał pod uwagę globalnie zdefiniowanych rozrzutów. Sprowadzać się to będzie do stworzenia podobwodu, który będzie miał wyłączone parametry kontrolujące mismatch i process (wartości tych parametrów dla podobwodów są traktowane lokalnie, więc nie trzeba się martwić o nadpisywanie wcześniej dodanych parametrów). Nazwa modelu powinna dokładnie opisywać element i jego funkcję, pamiętając o konwencji nazywania plików dla danej biblioteki - `sky130_fd_pr__` to fragment wskazujący na bibliotekę, a `pfet_01v8` to nazwa modelu. Można zweryfikować jak wyglądają definicje instancji sprawdzając netlistę (Shift + N, następnie `Simulation -> Edit Netlist`).

Dzięki zachowaniu tej składni, po dodaniu fragmentu definiującego podschemat, wystarczy zamienić parametr `model` danego elementu bez tworzenia nowych symboli dla modelu.

1.7. W naszych następnych symulacjach chcemy analizować rozrzuty pochodzące jedynie od tranzystora M1. Ponieważ domyślnie włączenie `.param mc_mm_switch = 1` aktywuje losowanie parametrów dla wszystkich elementów na schemacie, aby zasymulować idealne źródło referencyjne `M_REF` (bez rozrzutów) stworzymy dla tego schematu wspomniany subcircuit - pamiętając o konwencji nazywania elementów w bibliotece.

```
* disable pfet_ideal from mismatch/process
.subckt sky130_fd_pr__pfet_ideal D G S B W=0.42 L=0.18
+ nf=1 ad='int((1 + 1)/2) * {W} / 1 * 0.29' as='int((1 + 2)/2) * {W} / 1 * 0.29'
+ pd='2*int((1 + 1)/2) * ({W} / 1 + 0.29)' ps='2*int((1 + 2)/2) * ({W} / 1 +
0.29)'
+ nrd='0.29 / {W}' nrs='0.29 / {W}' sa=0 sb=0 sd=0 mult=1
.param mc_mm_switch=0
.param mc_pr_switch=0
```

```
XIDEAL D G S B sky130_fd_pr__pfet_01v8 W={W} L={L}
+ nf={nf} ad={ad} as={as} pd={pd} ps={ps}
+ nrd={nrd} nrs={nrs} sa={sa} sb={sb} sd={sd} mult={mult}
.ends
```

Ten fragment można dodać do któregoś z bloków kodu, ale zalecane jest dodanie go do bloku TT_MODELS gdzie oprócz dodawania plików cornerowych będziemy także uwzględniać efekty mismatchu i procesu. Zamień teraz model tranzystora M_REF z `pfet_01v8` na `pfet_ideal`. Jeżeli do tej pory wszystko poprawnie zostało zrobione - symulacja będzie uwzględniać rozrzuty wyłącznie pochodzące od tranzystora M1.

1.8. Przeprowadź analizy Monte Carlo (100 prób dla każdego z przypadków) dla wariantów prądów oraz wymiarów tranzystorów M_REF i M1 podanych uprzednio (pamiętaj o uwzględnieniu wyłącznie M1 w rozrzutach).

Uzyskane wyniki σ/AVG (sigma/wartość średnia) zanotuj w sprawozdaniu. Uzupełnij wszystkie brakujące pola w tabeli.

1.9. Odpowiedz na poniższe pytania:

- dlaczego dla tych samych wymiarów tranzystorów a różnych prądów znacząco różnią się wartości σ/AVG ?
- dlaczego dla tych samych zarówno powierzchni tranzystorów jak i ich prądów wartości σ/AVG różnią się pomiędzy sobą?
- wskaż dwa z analizowanych przypadków, które dowodzą różnych kontrybutorów rozrzutów (kontrybucja od napięcia progowego, kontrybucja od współczynnika prądowego).

1.10. Sprawdź stopień kontrybucji rozrzutów prądu wyjściowego (σ/AVG) dla trzech przypadków (rozważ tylko przypadek 2 / 5 μm) oraz $I_{\text{DAC}} = 10 \mu\text{A}$; Tylko pochodzące z mismatchu

`.mc_mm_switch=1 .mc_pr_switch=0`):

- uwzględniamy tylko rozrzuty tranzystora M1 (model M_REF ustaw jako `pfet_ideal`),
- uwzględniamy tylko rozrzuty tranzystora M_REF (model M1 ustaw jako `pfet_ideal`),
- uwzględniamy rozrzuty obu tranzystorów (model obu tranzystorów to `pfet_01v8`).

Wyniki zapisz w sprawozdaniu. Czy są one zgodne z Twoimi przypuszczeniami? Dlaczego?

1.11. Powtórz symulacje z poprzedniego punktu uwzględniając proces.

Uwzględnij rozrzuty Process i Mismatch (`.mc_mm_switch=1 .mc_pr_switch=1`). Wyjaśnij występujące różnice.

2. Schemat 5-bitowego DAC-a

TBD

3. Layout 5-bitowego DAC-a

TBD

4. Possible simulation errors

TODO LIST

1. Znaleźć sposób na uwzględnienie rozrzutów tylko pojedynczego tranzystora i uzupełnić punkty 1.5 - 1.7 (znalazłem sposób - trzeba uzupełnić instrukcję) - zrobione (chyba dobrze)
 - To co Defaultowo jest przez `SKY130->Add models symbol` może być zmienione poprzez modyfikację `/foss/pdks/sky130A/libs.tech/xschem/xschemrc`, ale to nie będzie permanentna zmiana, a wyłącznie na obecną sesję dockera, do stałej zmiany można dodać plik `/foss/designs/xschemrc` **POD WARUNKIEM że będziemy startować każdy schematic z folderu /foss/designs co przy dużej ilości podfolderów i plików, czy kilkusobowej pracy nad jednym projektem, może być niewygodne (trzeba być zapoznanym z drzewem folderów)**
 - dla bardzo małych prądów, mogą powstawać niedokładności między dużą ilością powtórzeń. Czemu? Może to przez niedokładności w modelowaniu? to nie jest problem tylko z .subckt, dla pętli prosto z sky130 też tak to wygląda
2. Przeprowadź analizę MC z punktu 1.8 i zapisz wyniki (kod do symulacji gotowy - ustawić M_REF jako mosfet idealny) - zrobione
3. Zaktualizuj polecenie 1.10 uwzględniając odpowiednie parametry i sposoby na rozrzucanie pojedynczego tranzystora - zrobione
4. Wykonaj symulacje zgodnie z punktem 1.11 - zrobione
5. **5BIT DAC SCHEMATIC** - zrobione
6. Stwórz symbol dla 5bit dac
7. Stwórz schemat symulacyjny
8. Zastanów się jak zrobić symulacje cornerowe dla każdego przypadku (chodzi o każdy corner w jednej symulacji - czy jest to wogóle możliwe)
9. LAYOUT TBD